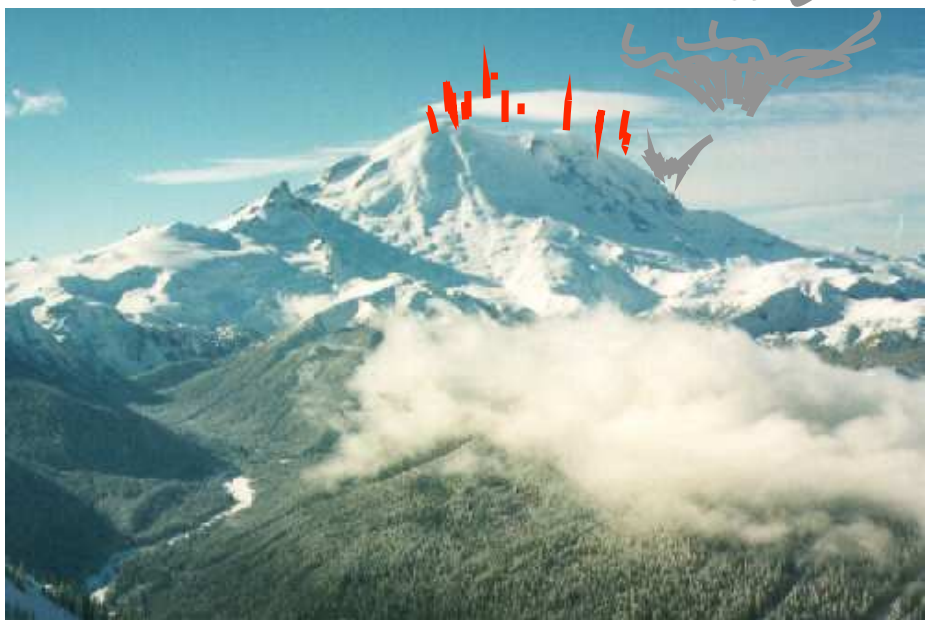
Quantum Computing

Lecture 4: Quantum Algorithms

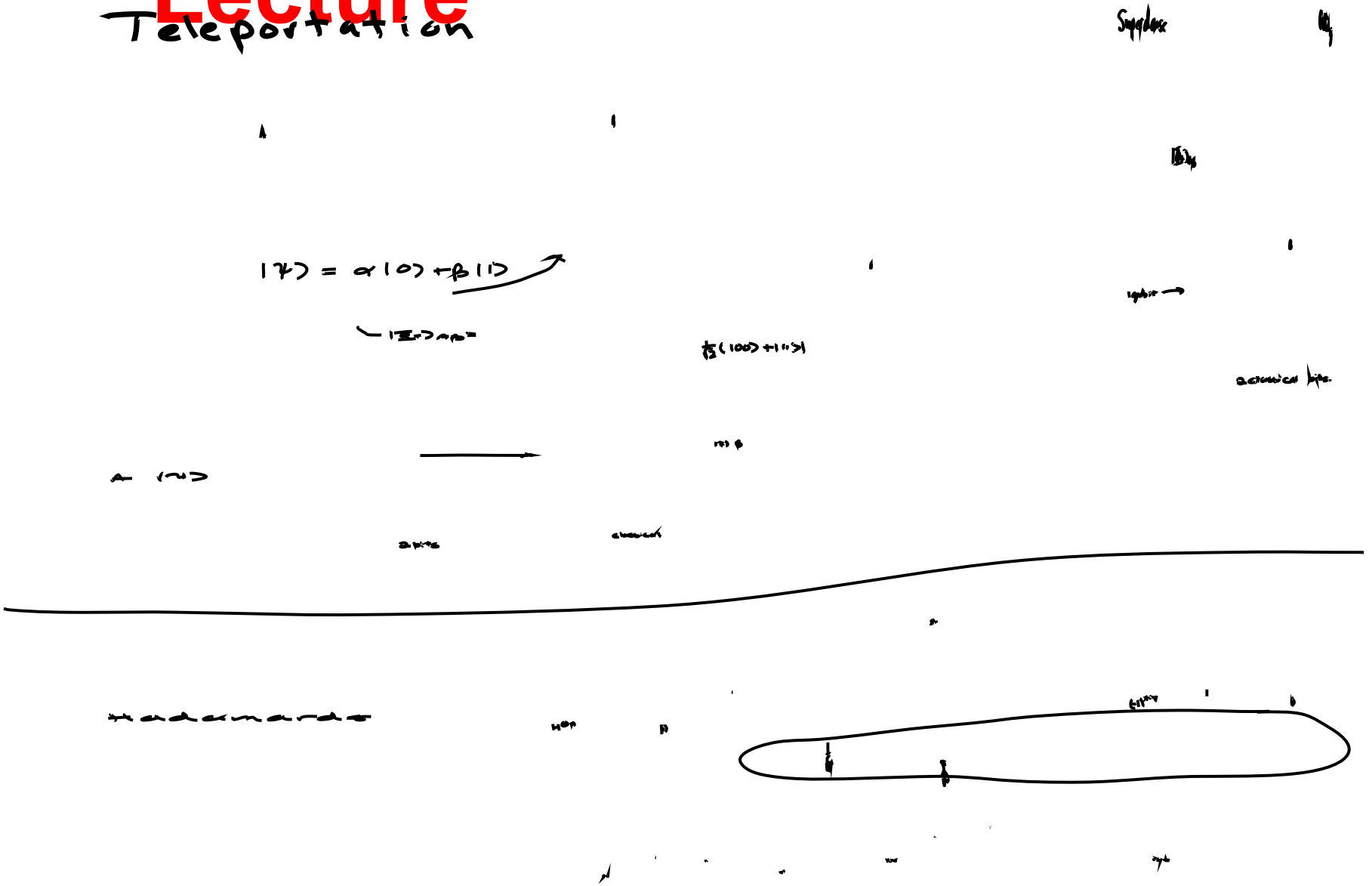
Dave Bacon

Department of Computer Science & Engineering
University of Washington



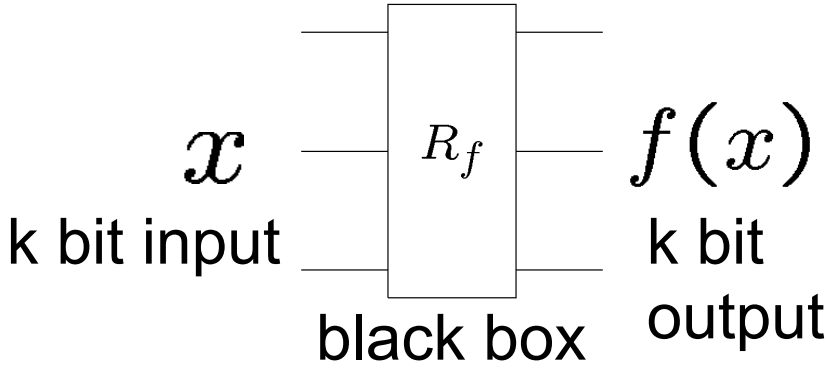


Summary of Last Lecture



Classical Promise Problem Query Complexity

Given: A black box which computes some function



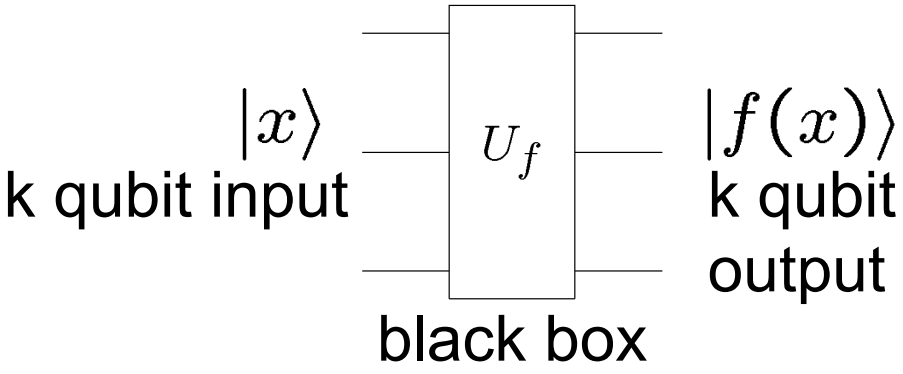
Promise: the function belongs to a set $\mathcal{S}$ which is a subset of all possible functions.

Properties: the set $\mathcal{S}$ can be divided into disjoint subsets $\mathcal{S}_1, \mathcal{S}_2, \ldots, \mathcal{S}_m$

Problem: What is the minimal number of times we have to use (query) the black box in order to determine which subset $\mathcal{S}_i$ the function belongs to?

Quantum Promise Query Complexity

Given: A quantum gate which, when used as a classical device computes a reversible function



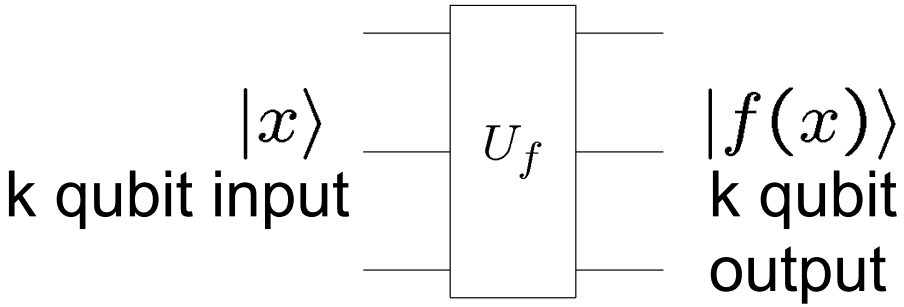
Promise: the function belongs to a set $\mathcal{S}$ which is a subset of all possible functions.

Properties: the set $\mathcal{S}$ can be divided into disjoint subsets $\mathcal{S}_1, \mathcal{S}_2, \ldots, \mathcal{S}_m$

Problem: What is the minimal number of times we have to use (query) the quantum gate in order to determine which subset $\mathcal{S}_i$ the function belongs to?

Functions

We can write the unitary



in outer product form as

$$U_f = \sum_{x=0}^{2^n-1} |f(x)\rangle\langle x|$$

so
tha

$$U_f|y\rangle = \left(\sum_{x=0}^{2^n-1} |f(x)\rangle\langle x|\right)|y\rangle$$

$$= \sum_{x=0}^{2^n-1} |f(x)\rangle\langle x|y\rangle = \sum_{x=0}^{2^n-1} |f(x)\rangle\delta_{y,x} = |f(y)\rangle$$

Functions

Note that the transform is unitary:

$$U_f^\dagger = \left(\sum_{x=0}^{2^n-1} |f(x)\rangle\langle x|\right)^\dagger = \sum_{x=0}^{2^n-1} (|f(x)\rangle\langle x|)^\dagger$$

$$= \sum_{x=0}^{2^n-1} |x\rangle\langle f(x)|$$

When

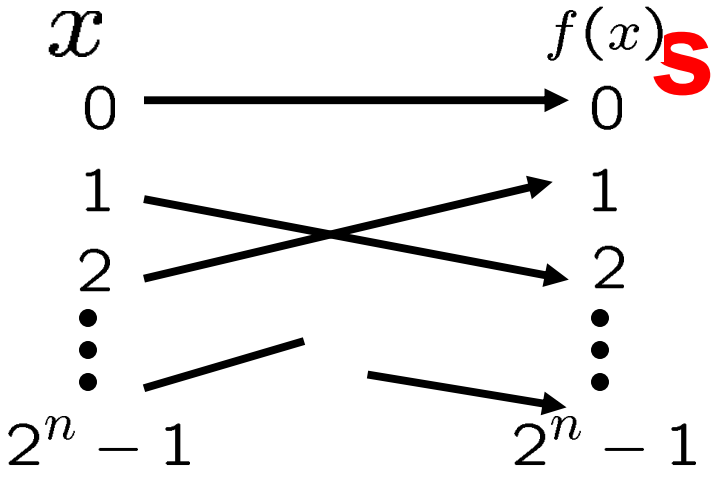
$$U_f U_f^\dagger = \left(\sum_{x=0}^{2^n-1} |f(x)\rangle\langle x|\right)\left(\sum_{y=0}^{2^n-1} |y\rangle\langle f(y)|\right)$$

$$= \sum_{x,y=0}^{2^n-1} |f(x)\rangle\langle x|y\rangle\langle f(y) = \sum_{x,y=0}^{2^n} |f(x)\rangle\langle f(y)|\delta_{x,y}$$

$$= \sum_{x=0}^{2^n-1} |f(x)\rangle\langle f(x)| = I$$

precisely when f(x) is one to one!

Functions



One to one

Example : Not one to one

$f(x) = 0 \quad x \in \{0, 1\}$

$$U_f = \sum_{x=0}^{1} |f(x)\rangle\langle x| = |0\rangle\langle 0| + |0\rangle\langle 1|$$

$$U_f^\dagger = |0\rangle\langle 0| + |1\rangle\langle 0|$$

$$U_f^\dagger U_f = (|0\rangle\langle 0| + |1\rangle\langle 0|)(|0\rangle\langle 0| + |0\rangle\langle 1|)$$

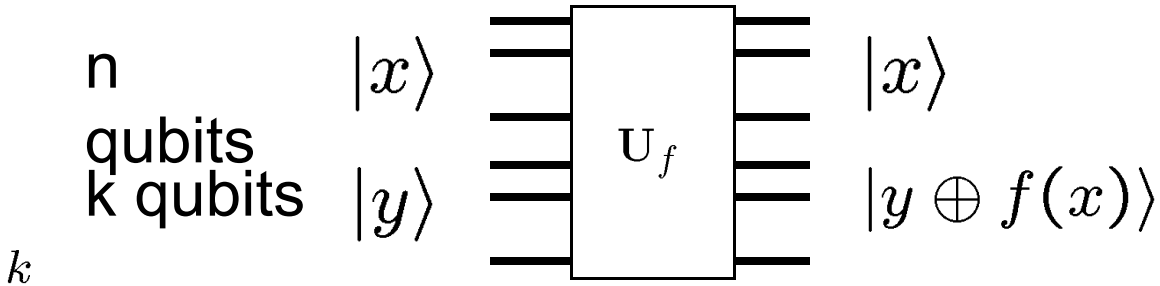
$$= |0\rangle\langle 0| + |0\rangle\langle 1| + |1\rangle\langle 0| + |1\rangle\langle 1| \neq I$$

An Aside on Functions

Generically we can compute a non-reversible function using the following trick:

function from n bits to k bits:

$$f : x \in \{0,1\}^n \rightarrow \{0,1\}^k$$



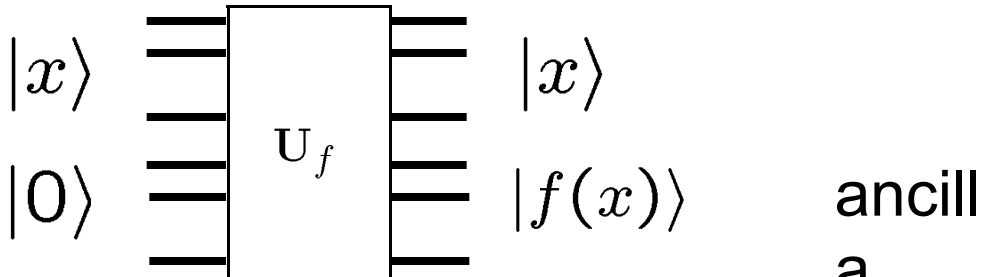
$$y, f(x) \in \{0,1\}^k$$

$$x \in \{0,1\}^n$$

$y \oplus f(x)$ is a bitwise exclusive or

$$y \oplus f(x) = (y_1 \oplus f(x)_1, y_2 \oplus f(x)_2, \ldots, y_k \oplus f(x)_k)$$

Such that, with proper input we can calculate f:



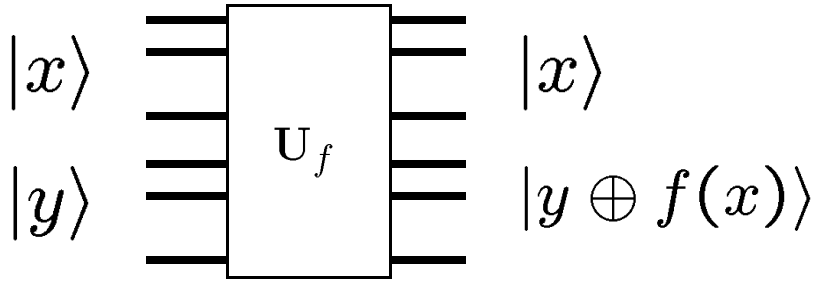
An Aside on Functions

function from n bits to k bits:

$$f : x \in \{0,1\}^n \rightarrow \{0,1\}^k$$

n qubits

k qubits



$$y, f(x) \in \{0,1\}^k$$

$$x \in \{0,1\}^n$$

$$U_f = \sum_{x,y=0}^{2^n-1} |x\rangle|y \oplus f(x)\rangle\langle x|\langle y|$$

$y \oplus f(x)$ is a bitwise exclusive or

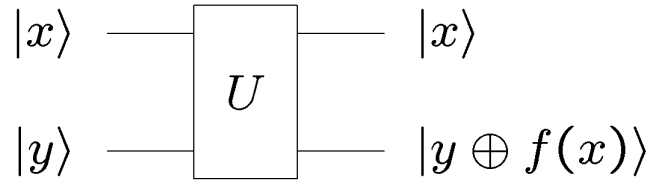
$$U_f^\dagger = \sum_{x,y=0}^{2^n-1} |x\rangle|y\rangle\langle x|\langle y \oplus f(x)|$$

$$U_f U_f^\dagger = \sum_{x,y=0}^{2^n-1} |x\rangle|y \oplus f(x)\rangle\langle x|\langle y| \sum_{x',y'=0}^{2^n-1} |x'\rangle|y'\rangle\langle x'|\langle y' \oplus f(x')|$$

$$= \sum_{x,y=0}^{2^n-1} |x\rangle|y \oplus f(x)\rangle\langle x|\langle y \oplus f(x)| = I$$

From This Perspective

$00 \to 00$	$00 \to 01$	$00 \to 00$	$00 \to 01$
$01 \to 01$	$01 \to 00$	$01 \to 01$	$01 \to 00$
$10 \to 10$	$10 \to 11$	$10 \to 11$	$10 \to 10$
$11 \to 11$	$11 \to 10$	$11 \to 10$	$11 \to 11$
"identity"	NOT 2nd bit	controlled-NOT	controlled-NOT + NOT 2nd bit

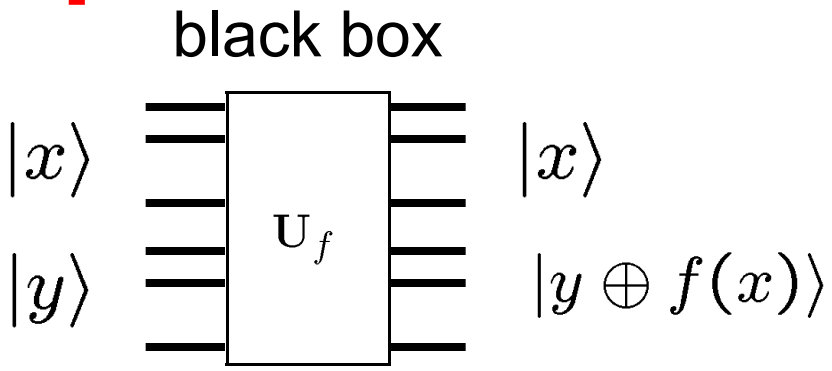


$f(x) = 0$	$f(x) = 1$	$f(x) = x$	$f(x) = \bar{x}$
constant functions		balanced functions	

Deutsch's problem is to distinguish constant from balanced

Query Complexities

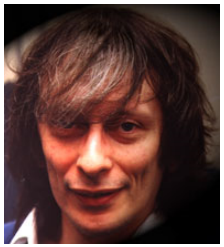
$$f : x \in \{0,1\}^n \rightarrow \{0,1\}^k$$



	probability of failure
Exact classical query complexity	0
Bounded error classical query complexity	$< \frac{1}{2}$
Exact quantum query complexity	0
Bounded error quantum query complexity	$< \frac{1}{2}$

Bounded error algorithms are allowed to fail with a bounded probability of failure.

Quantum Algorithms



David Deutsch



Richard Jozsa

1992: Deutsch-Jozsa Algorithm

Exact classical q. complexity: $2^{n-1}+1$

Bounded error classical q. complexity: $O(1)$

Exact quantum q. complexity: 1



Umesh Vazirani



Ethan Bernstein

1993: Bernstein-Vazirani Algorithm (non-recursive)

Exact classical q. complexity: n

Bounded error classical q. complexity: $\Omega(n)$

Exact quantum q. complexity: 1

Quantum Algorithms



Umesh Vazirani



Ethan Bernstein

1993: Bernstein-Vazirani Algorithm (recursive)

Bounded error classical q. complexity: $n^{\Omega(\log n)}$

Exact quantum q. complexity: $O(n \log n)$

(first super-polynomial separation)



Dan Simon

1994: Simon's Algorithm

Bounded error classical q. complexity: $\Omega(2^{n/2})$

Bounded error quantum q. complexity: $O(n)$

(first exponential separation)

Generalizing Simon's algorithm, in 1994, Peter Shor was able to derive an algorithm for efficiently factoring and discrete log!

The Factoring Firestorm



Peter Shor 1994

188198812920607963383697239461650439807163563379417382700763356422988859715234665485319060606504743045317388011303396716199623212057340318795506569622130516875930765025705 9

=

3980750864240649373971255005503864911990643623425267084063851895759463889572617685833172727214610743530253622307197304822463291469530209711645985217113052071125636359039752 7

×

Best classical algorithm takes time $O(\exp(n^{1/3}))$

Shor's quantum algorithm takes time $O(n^2 \log n)$

An efficient algorithm for factoring **breaks** the RSA public key cryptosystem

Deutsch-Jozsa Problem

Given: A function with n bit strings as input and one bit as output

$$f : x \in \{0,1\}^n \rightarrow \{0,1\}$$

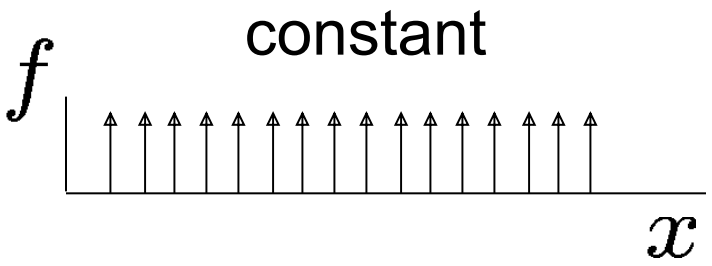
(this will be a non-reversible function)

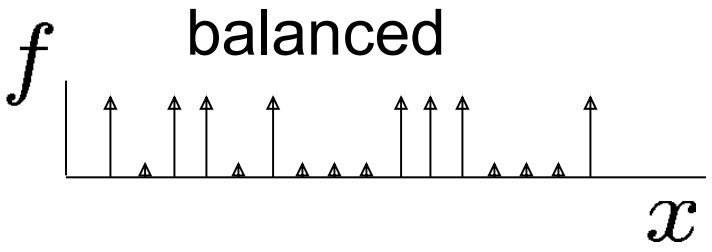
Promise: The function is either constant or balance.

constant function: $\forall x \in \{0,1\}^n, f(x) = b$

balanced function: $\forall x \in \mathcal{H}, f(x) = 0, \forall x \notin \mathcal{H}, f(x) = 1$

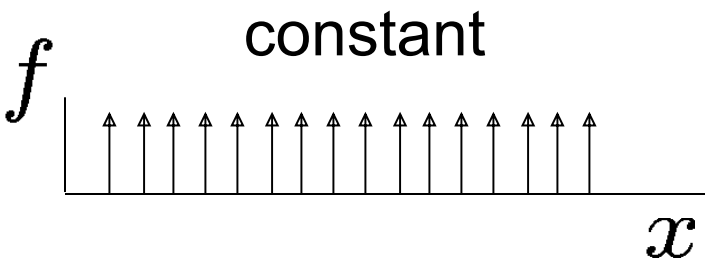
$$\mathcal{H} \subset \{0,1\}^n, |\mathcal{H}| = 2^{n-1}$$

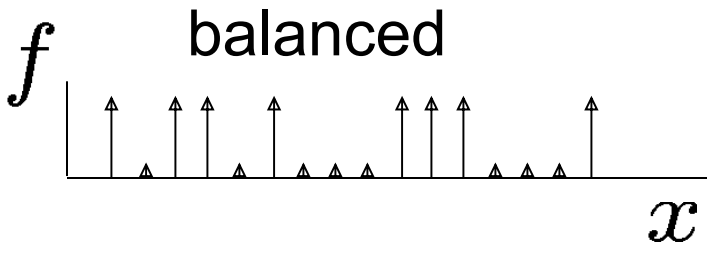




Problem: determine whether the function is constant or balanced.

Classical Deutsch-Jozsa



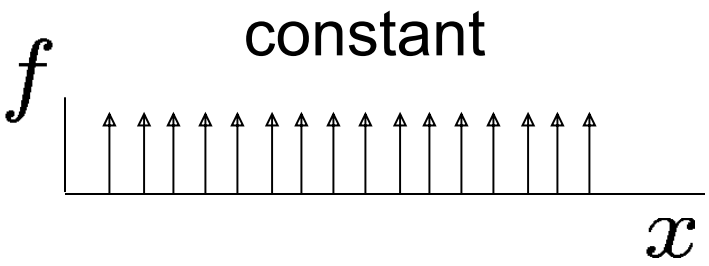


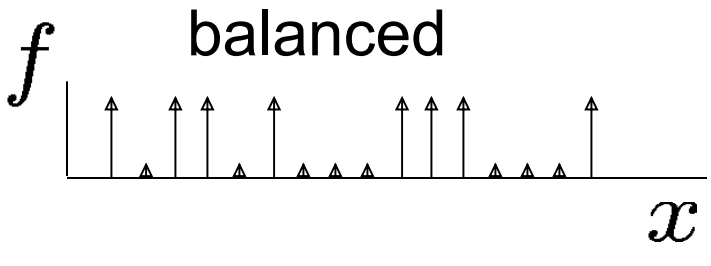
Problem: determine whether the function is constant or balanced.

No failure allowed: we need to query in the worst case $2^{n-1}+1$ values of f to distinguish between constant and balanced

Exact classical q. complexity: $2^{n-1}+1$

Classical Deutsch-Jozsa





Problem: determine whether the function is constant or balanced.

Bounded error: Query two different random values of the function.

If they are equal, guess constant.
Otherwise, guess balanced.

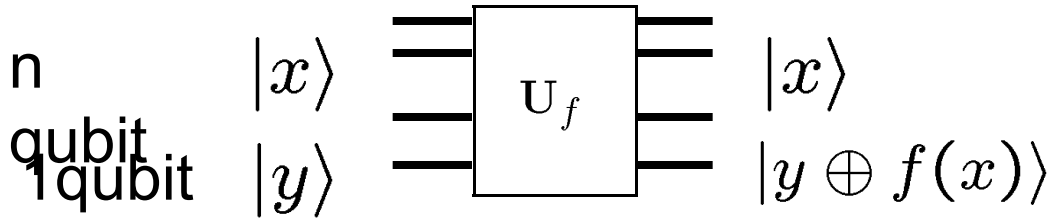
$$Pr(fail) = \frac{2^{n-1} - 1}{2^n - 1} < \frac{1}{2}$$

Bounded error classical q. complexity: $O(1)$

Quantum Deutsch-Jozsa

Given: A quantum gate on n+1 qubits strings which calculates the promised f

$$f : x \in \{0,1\}^n \rightarrow \{0,1\}$$



$$y, f(x) \in \{0,1\}$$

$$x \in \{0,1\}^n$$

$$|x\rangle \otimes |y\rangle \xrightarrow{\mathbf{U}_f} |x\rangle \otimes |y \oplus f(x)\rangle$$

Trick 1: Phase Kickback



$$|x\rangle \otimes |y\rangle \overset{\mathbf{U}_f}{\rightarrow} |x\rangle \otimes |y \oplus f(x)\rangle$$

Input a superposition over second register:

$$|x\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \overset{\mathbf{U}_f}{\rightarrow} |x\rangle \otimes \frac{1}{\sqrt{2}}(|f(x)\rangle - |\bar{f}(x)\rangle)$$

$$f(x) = 0 \quad |x\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

$$f(x) = 1 \quad -|x\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Function is computed into phase:

$$|x\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \overset{\mathbf{U}_f}{\rightarrow} (-1)^{f(x)}|x\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Trick 2: Hadamarding Qubits





$$|x\rangle \;\boxed{H}\; \frac{1}{\sqrt{2}}(|0\rangle + (-1)^x|1\rangle)$$

$$|x_1\rangle \;\boxed{H}$$

$$|x_2\rangle \;\boxed{H}$$

$$|x_3\rangle \;\boxed{H}$$

$$\vdots$$

$$|x_n\rangle \;\boxed{H}$$

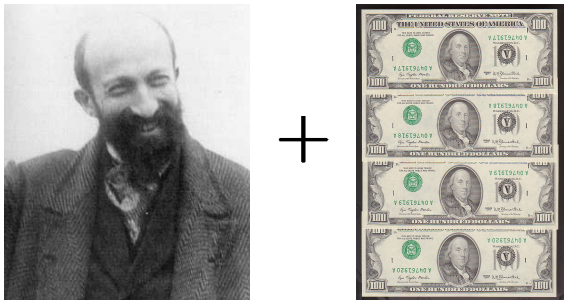
$$|x\rangle = |x_1, x_2, \ldots, x_n\rangle$$

$$\frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} (-1)^{y \cdot x} |y\rangle$$

$$y \cdot x = y_1 x_1 \oplus y_2 x_2 \oplus \cdots \oplus y_n x_n$$

Note: $H^{\otimes n}|0\rangle = \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} |y\rangle$ and $H^{\otimes n} \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} |y\rangle = |0\rangle$

Tricks 1 and 2 Together



n qubits

$|0\rangle$ — H

$|0\rangle$ — H

$|0\rangle$ — H

$|0\rangle$ — H

$|1\rangle$ — H

U_f

$$\frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} (-1)^{f(y)} |y\rangle \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)$$

$$H^{\otimes n}|0\rangle \otimes H|1\rangle = \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} |y\rangle \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)$$

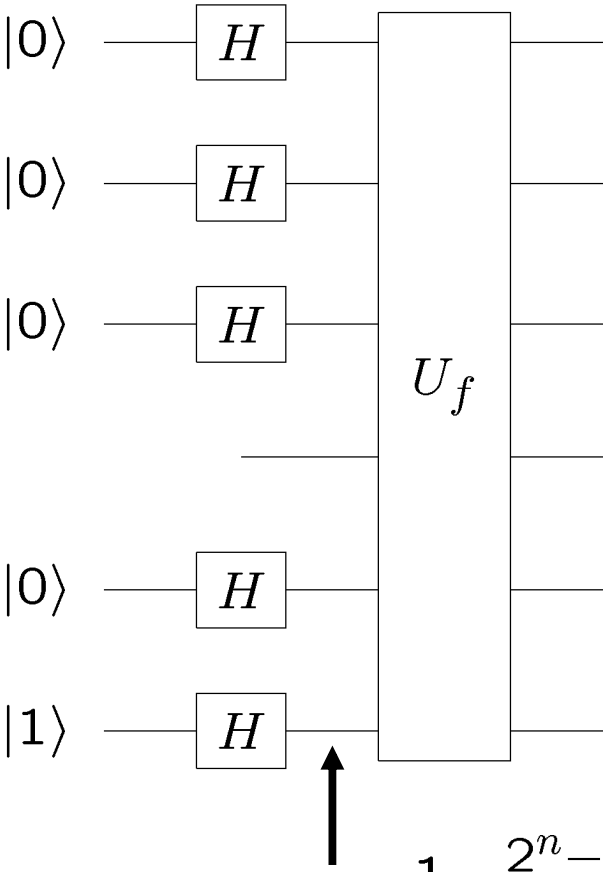
Tricks 1 and 2 Together



+



n qubits



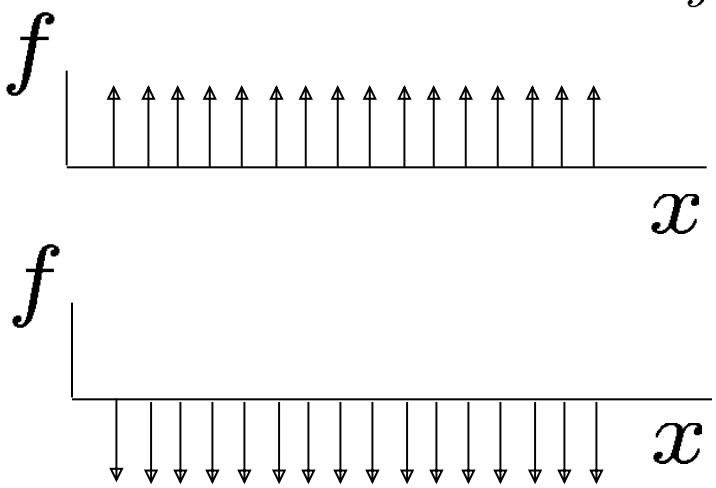
$$\frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} (-1)^{f(y)} |y\rangle$$

$$\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

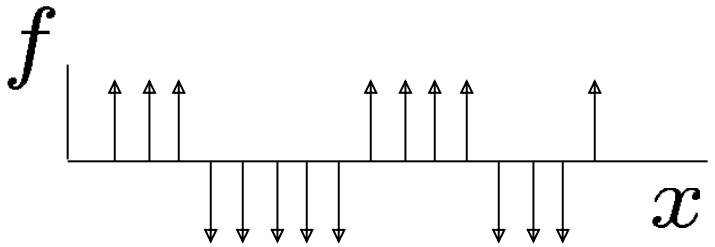
$$H^{\otimes n}|0\rangle \otimes H|1\rangle = \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} |y\rangle \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Function in the Phase

$$\frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} (-1)^{f(y)} |y\rangle$$



constant



balanced

$$\frac{(-1)^{f(0)}}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} |y\rangle$$

Function in the Phase

When the function is constant:

$$H^{\otimes n}\left[\frac{(-1)^{f(0)}}{\sqrt{2^n}}\sum_{y=0}^{2^n-1}|y\rangle\right] = (-1)^{f(0)}|0\rangle$$

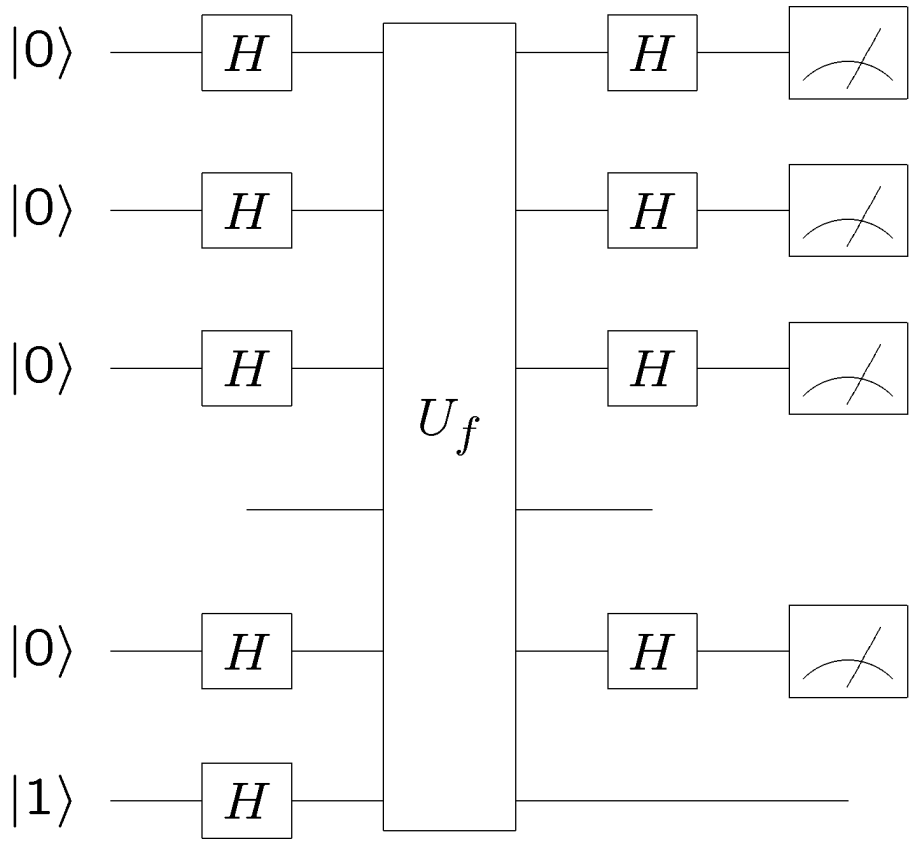
When the function is balanced:

$$H^{\otimes n}\left[\frac{1}{\sqrt{2^n}}\sum_{y=0}^{2^n-1}(-1)^{f(y)}|y\rangle\right] = \frac{1}{2^n}\sum_{x,y=0}^{2^n-1}(-1)^{f(y)+y\cdot x}|x\rangle$$

amplitude in zero state

$$\langle 0|H^{\otimes n}\left[\frac{1}{\sqrt{2^n}}\sum_{y=0}^{2^n-1}(-1)^{f(y)}|y\rangle\right] = \frac{1}{2^n}\sum_{y=0}^{2^n-1}(-1)^{f(y)} = 0$$

Quantum Deutsch-Jozsa



n qubits

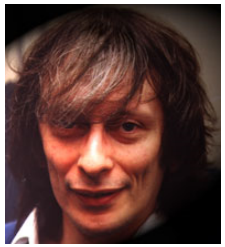
r

If function is constant, r is always 0.

If function is balanced, r is never 0.

Distinguish constant from balanced using one quantum query

Deutsch-Jozsa



David Deutsch



Richard Jozsa

1992: Deutsch-Jozsa Algorithm

Exact classical q. complexity: $2^{n-1}+1$

Bounded error classical q. complexity: $O(1)$

Exact quantum q. complexity: 1

Bernstein-Vazirani Problem

Given: A function with n bit strings as input and one bit as output

$$f : x \in \{0,1\}^n \rightarrow \{0,1\}$$

Promise: The function is of the form

$$f(x) = (a \cdot x) \oplus b \qquad a \in \{0,1\}^n \quad b \in \{0,1\}$$

$$y \cdot x = y_1 x_1 \oplus y_2 x_2 \oplus \cdots \oplus y_n x_n$$

Problem: Find the n bit string a

Classical Bernstein-Vazirani

Given: A function with n bit strings as input and one bit as output

$$f : x \in \{0,1\}^n \rightarrow \{0,1\}$$

Promise: The function is of the form

$$f(x) = (a \cdot x) \oplus b \qquad a \in \{0,1\}^n \quad b \in \{0,1\}$$

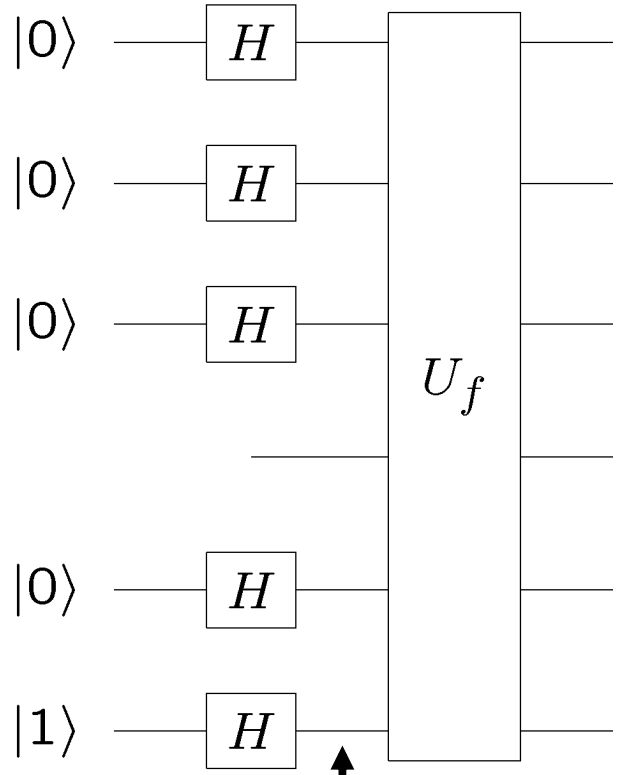
$$y \cdot x = y_1 x_1 \oplus y_2 x_2 \oplus \cdots \oplus y_n x_n$$

Problem: Find the n bit string a

Notice that the querying f yields a single bit of information. But we need n bits of information to describe a.

Bounded error classical q. complexity: $\Omega(n)$

Quantum Bernstein-Vazirani



n qubits

$$\frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} (-1)^{f(y)} |y\rangle$$

$$= \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} (-1)^{a \cdot y + b} |y\rangle$$

$$\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

$$H^{\otimes n}|0\rangle \otimes H|1\rangle = \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} |y\rangle \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Hadamard It!

$$H^{\otimes n}\left[\frac{1}{\sqrt{2^n}}\sum_{y=0}^{2^n-1}(-1)^{a\cdot y+b}|y\rangle\right]=\frac{1}{2^n}\sum_{x,y=0}^{2^n-1}(-1)^{a\cdot y+b}(-1)^{x\cdot y}|x\rangle$$

$$=\frac{(-1)^b}{2^n}\sum_{x=0}^{2^n-1}\left(\sum_{y=0}^{2^n-1}(-1)^{a\cdot y+x\cdot y}\right)|x\rangle$$

$$\sum_{y=0}^{2^n-1}(-1)^{a\cdot y+x\cdot y}=\sum_{y_1,\ldots,y_n=0}^{1}(-1)^{a_1y_1+\cdots+a_ny_n+x_1y_1+\cdots+x_ny_n}$$

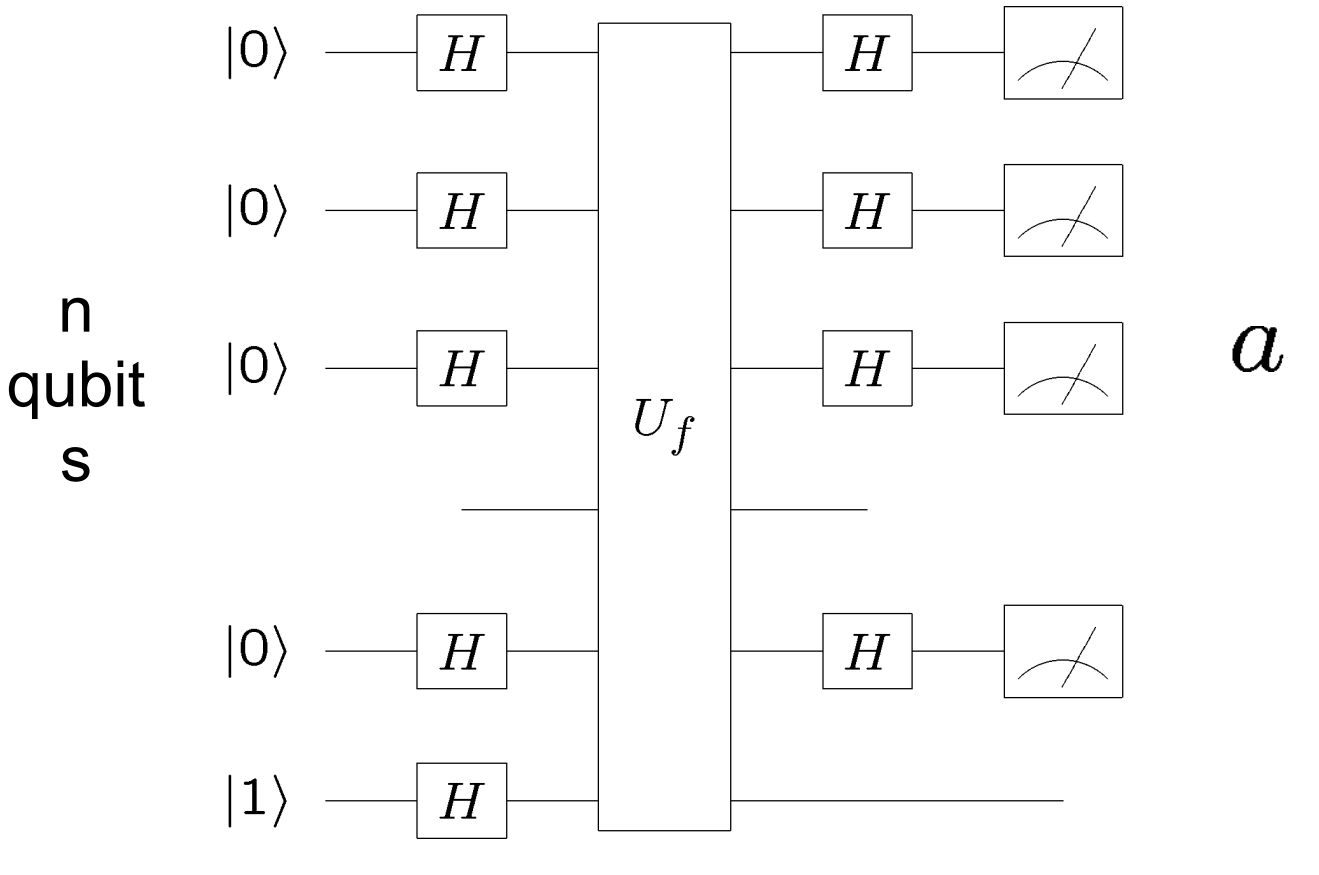
$$=\sum_{y_1=0,1}\cdots\sum_{y_n=0,1}(-1)^{a_1y_1+\cdots+a_ny_n+x_1y_1+\cdots+x_ny_n}$$

$$=((-1)^0+(-1)^{a_1+x_1})\sum_{y_2=0,1}\cdots\sum_{y_n=0,1}(-1)^{a_2y_2+\cdots+a_ny_n+x_2y_2+\cdots+x_ny_n}$$

$$=2^n\delta_{a_1,x_1}\delta_{a_2,x_2}\cdots\delta_{a_n,x_n}=2^n\delta_{a,x}$$

$$H^{\otimes n}\left[\frac{1}{\sqrt{2^n}}\sum_{y=0}^{2^n-1}(-1)^{a\cdot y+b}|y\rangle\right]=(-1)^b\sum_{x=0}^{2^n}\delta_{a,x}|x\rangle=(-1)^b|a\rangle$$

Quantum Bernstein-Vazirani



We can determine a using only a single quantum query!

Bernstein-Vazirani



Umesh Vazirani



Ethan Bernstein

1993: Bernstein-Vazirani Algorithm (non-recursive)

Exact classical q. complexity: n

Bounded error classical q. complexity: $\Omega(n)$

Exact quantum q. complexity: 1

APS Journals Physical Review Letters
AMERICAN PHYSICAL SOCIETY

Fast Quantum Algorithm for Numerical Gradient Estimation

Stephen P. Jordan*

Physics Department, MIT, 77 Massachusetts Avenue, Cambridge, Massachusetts 02139, USA
(Received 25 May 2004; revised manuscript received 3 January 2005; published 28 July 2005)

Given a black box for f, a smooth real scalar function of d real variables, one wants to estimate ∇f at a given point with n bits of precision. On a classical computer this requires a minimum of $d+1$ black box queries, whereas on a quantum computer it requires only one query regardless of d. The number of bits of precision to which f must be evaluated matches the classical requirement in the limit of large n.

DOI: 10.1103/PhysRevLett.95.050501 PACS numbers: 03.67.Lx

We investigate the query complexity of numerically estimating the gradient of a black box function $f: R^d \rightarrow R$ at a given point. We find that gradients can be estimated on a quantum computer using a single black box query whereas we require at least $d+1$ queries classically. The algorithm which achieves this can be viewed as a generalization of the Bernstein-Vazirani [1] algorithm, which has been described in other contexts [2–5]. The black box in this algorithm was always previously described as evaluating a function over the integers rather than approximating a continuous function with finite precision. In [2], the question as to whether the algorithm could be adapted for any task of practical interest was presented as an open problem, which this paper resolves by showing a speedup for gradient estimation, which is a fundamental operation in many numerical calculations.

Bernstein-Vazirani



Umesh Vazirani



Ethan Bernstein

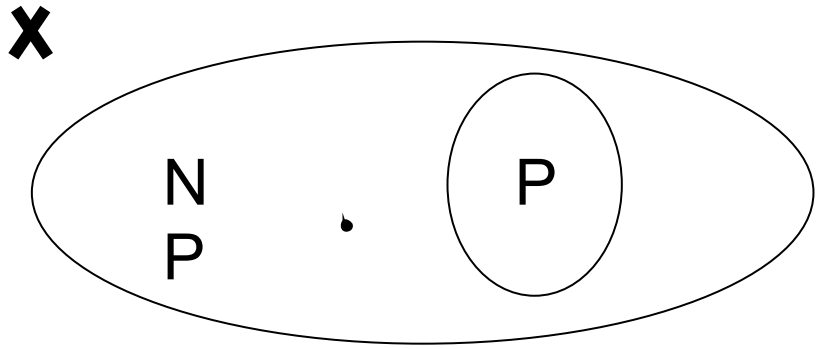
1993: Bernstein-Vazirani Algorithm (recursive)

Bounded error classical q. complexity: $n^{\Omega(\log n)}$

Exact quantum q. complexity: $O(n \log n)$

(first super-polynomial separation)

RFS (recursive Fourier sampling)



Simon's Problem

(is that nobody does what Simon says)

Given: A function with n bit strings as input and one bit as output

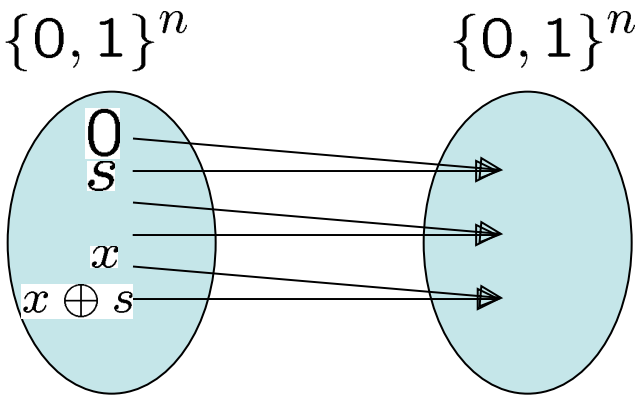
$$f : x \in \{0,1\}^n \rightarrow \{0,1\}^n$$

Promise: The function is guaranteed to satisfy

$$f(x) = f(y) \Leftrightarrow y = x \oplus s, s \neq 0$$

$$x \oplus s = (x_1 \oplus s_1, x_2 \oplus s_2, \ldots, x_n \oplus s_n)$$

Problem: Find the n bit string $s \neq 0$



Classical Simon's Problem

Promise: The function is guaranteed to satisfy

$$f(x) = f(y) \Leftrightarrow y = x \oplus s, s \neq 0$$

$$x \oplus s = (x_1 \oplus s_1, x_2 \oplus s_2, \ldots, x_n \oplus s_n)$$

Suppose we start querying the function and build up a list of the pairs $(x_\alpha, f(x_\alpha))$

If we find $x_\alpha \neq x_\beta$ such that $f(x_\alpha) = f(x_\beta)$ then we solve the problem:

$$f(x_\alpha) = f(x_\beta) \Rightarrow x_\alpha = s \oplus x_\beta$$

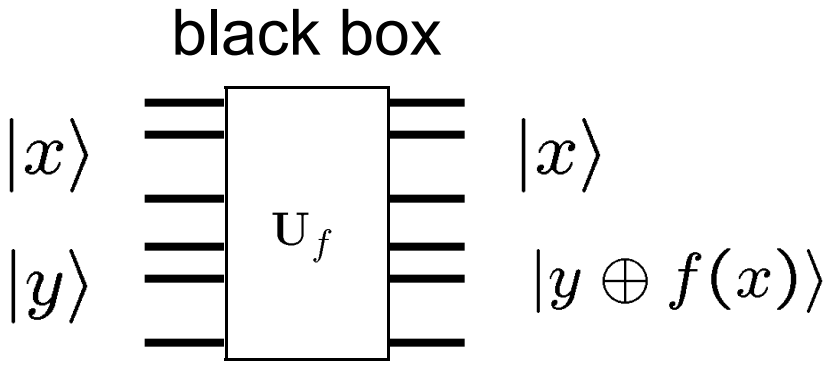
$$s = x_\alpha \oplus x_\beta$$

But suppose we start querying the function m times….

Probability of getting a matching pair $\approx \frac{\binom{m}{2}}{2^n} = \frac{m(m-1)}{2^{n+1}}$

Bounded error query complexity: $m = O\left(2^{\frac{n}{2}}\right)$

Quantum Simon's Problem



Unlike previous problems, we can't use the phase kickback trick because there is no structure in the function.

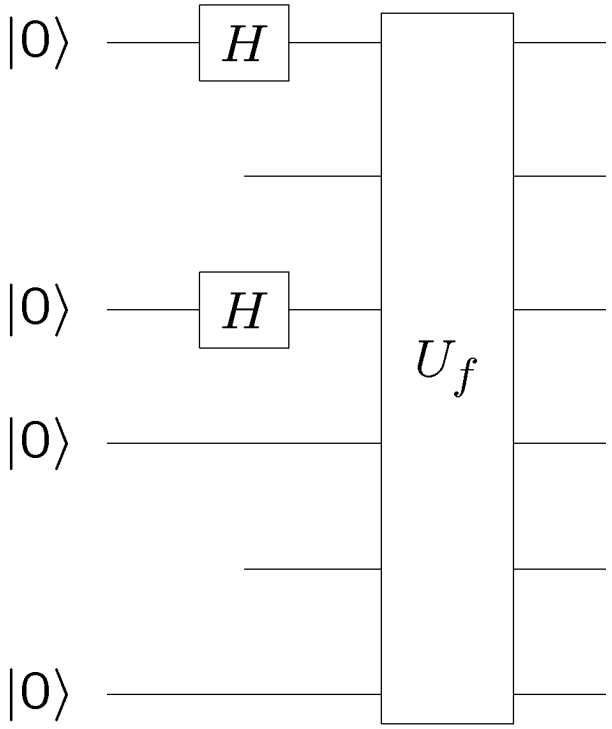
Charge ahead:



Quantum Simon's Problem

n qubits

n qubits



$$\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes |0\rangle$$

$$\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes |f(x)\rangle$$

Quantum Simon's Problem

Measure the second register

$$\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes |f(x)\rangle$$

Using the promise on the function

$$f(x) = f(y) \Leftrightarrow y = x \oplus s, s \neq 0$$

This implies that after we measure, we have the state

$$\frac{1}{\sqrt{2}}(|x\rangle + |x \oplus s\rangle) \otimes |f(x)\rangle$$

For random uniformly distributed $x \in \{0,1\}^n$

uniformly distributed = all strings equally probable

Measuring this state at this time does us no good….

Quantum Simon's Problem

$$\frac{1}{\sqrt{2}}(|x\rangle + |x \oplus s\rangle)$$

Measuring this state in the computational basis at this time does us no good….

For random uniformly distributed $x \in \{0,1\}^n$

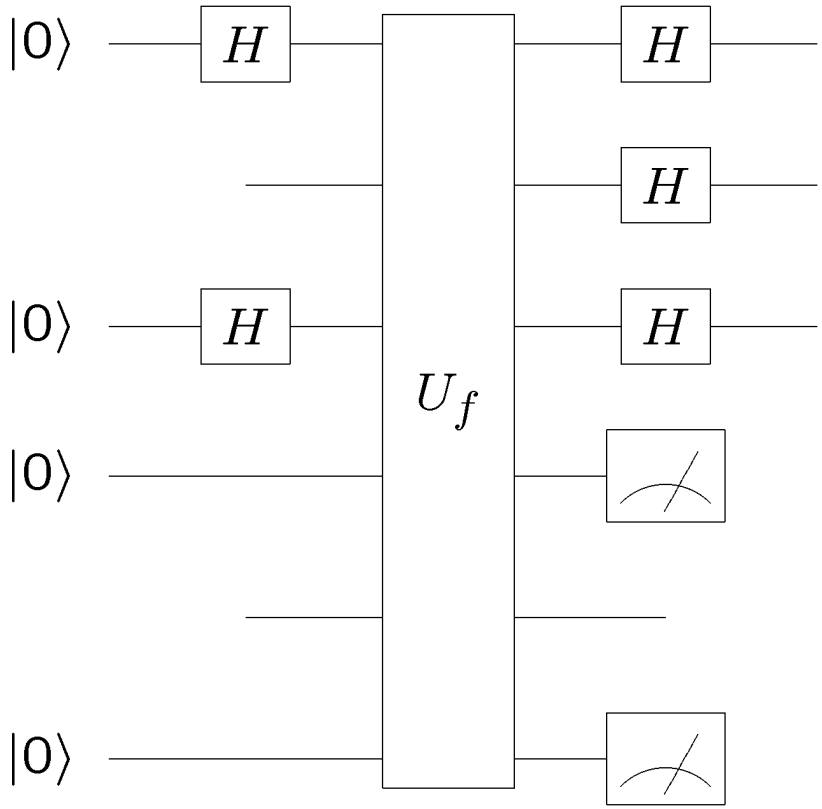
Measurement yields either

$|x\rangle$ or $|x \oplus s\rangle$

But we don't know x, so we can't use this to find s.

Quantum Simon's Problem

n qubits



n qubits

$$H^{\otimes n}\frac{1}{\sqrt{2}}(|x\rangle+|x\oplus s\rangle) = \frac{1}{\sqrt{2^{n+1}}}\sum_{y=0}^{2^n-1}((-1)^{y\cdot x}+(-1)^{y\cdot(x\oplus s)})|y\rangle$$

Quantum Simon's Problem

$$H^{\otimes n}\frac{1}{\sqrt{2}}(|x\rangle+|x\oplus s\rangle) = \frac{1}{\sqrt{2^{n+1}}}\sum_{y=0}^{2^n-1}((-1)^{y\cdot x}+(-1)^{y\cdot(x\oplus s)})|y\rangle$$

$$y\cdot x = y_1x_1 \oplus y_2x_2 \oplus \cdots \oplus y_nx_n$$

$$y\cdot(x\oplus s) = y_1(x_1\oplus s_1)\oplus y_2(x_2\oplus s_2)\oplus\cdots\oplus y_n(x_n\oplus s_n) = (y\cdot x)\oplus(y\cdot s)$$

$$= \frac{1}{\sqrt{2^{n+1}}}\sum_{y=0}^{2^n-1}(-1)^{y\cdot x}(1+(-1)^{y\cdot s})|y\rangle$$

Measuring this state, we obtain uniformly distributed random values of y such that

$$y\cdot s = 0 \mod 2$$

If $y \neq 0$ we have eliminated the possible values of s by half

Quantum Simon's Problem

$$y \cdot s = 0 \mod 2$$

$$y \cdot s = y_1 s_1 \oplus y_2 s_2 \oplus \cdots \oplus y_n s_n = 0$$

On values of y_i which are 0, this doesn't restrict s_i

On values of y_i which are 1, the corresponding s_i must XOR to 0. This restricts the set of possible s's by half.

Example: $n = 3$

$$y = 3_{10} = 011_2$$

$$(011) \cdot s = 0 s_1 \oplus 1 s_2 \oplus 1 s_3 = 0$$

$$s_2 \oplus s_3 = 0$$

possible s's: 000 011 100 111

(Z2)n Vectors

If single run eliminates half, multiple runs….how to solve?

Think about the bit strings x as vectors in $\mathbb{Z}_2^n$

vectors in $\mathbb{Z}_2^n$

$$x = (x_1, x_2, \ldots, x_n) \qquad x_i \in \{0, 1\}$$

We can add these vectors:

$$x = (x_1, x_2, \ldots, x_n)$$
$$y = (y_1, y_2, \ldots, y_n)$$
$$x + y = (x_1 + y_1, x_2 + y_2, \ldots, x_n + y_n)$$

Where all additions are done module 2

(Z2)n Vectors

Example:

$$x = (0, 1, 1, 0)$$

$$y = (1, 0, 1, 0)$$

$$x + y = (0 + 1, 1 + 0, 1 + 1, 0 + 0) = (1, 1, 0, 0)$$

We can multiply these vectors by a scalar in $\mathbb{Z}_2$

$$bx = (bx_1, bx_2, \ldots, bx_n) \quad b \in \{0, 1\} = \mathbb{Z}_2$$

Example:

$$b = 1$$

$$x = (0, 1, 1, 0)$$

$$bx = (0, 1, 1, 0)$$

(Z2)n Vectors

dot product of vectors in $\mathbb{Z}_2^n$

$$x = (x_1, x_2, \ldots, x_n)$$

$$y = (x_1, x_2, \ldots, x_n)$$

$$x \cdot y = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n \bmod 2$$

$$x \cdot y \in \mathbb{Z}_2$$

Example:

$$x = (0, 1, 1, 0)$$

$$y = (1, 0, 1, 0)$$

$$x \cdot y = 0 \times 1 + 1 \times 0 + 1 \times 1 + 0 \times 0 \bmod 2 = 1$$

(Z2)n Vectors

vectors in $\mathbb{Z}_2^n$

$$x = (x_1, x_2, \ldots, x_n)$$

one possible basis:

$$v_1 = (1, 0, \ldots, 0)$$
$$v_2 = (0, 1, \ldots, 0)$$
$$\vdots$$
$$v_n = (0, 0, \ldots, n)$$

$$x = x_1 v_1 + x_2 v_2 + \cdots + x_n v_n$$
$$= x_1(1, 0, \ldots, 0) + x_2(0, 1, \ldots, 0) + \cdots + x_n(0, 0 \ldots, 1)$$
$$= (x_1, 0, \ldots, 0) + (0, x_2, \ldots, 0) + \cdots + (0, 0 \ldots, x_n)$$

(Z2)n Vectors

vectors in $\mathbb{Z}_2^n$

$$x = (x_1, x_2, \ldots, x_n)$$

But we can expand in about a different set of vectors

$$w_1, w_2, \ldots, w_n$$

Example $n = 2$:

$$w_1 = (0, 1)$$

$$w_2 = (1, 1)$$

$$(x_1 + x_2)w_1 + x_1 w_2 = (x_1 + x_2)(0, 1) + x_1(1, 1)$$

$$= (0, x_1 + x_2) + (x_1, x_1) = (x_1, x_1 + x_2 + x_1)$$

$$= (x_1, x_2)$$

(Z2)n Vectors

vectors in $\mathbb{Z}_2^n$

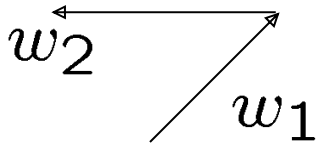
$$x = (x_1, x_2, \ldots, x_n)$$

But we can expand in about a different set of vectors

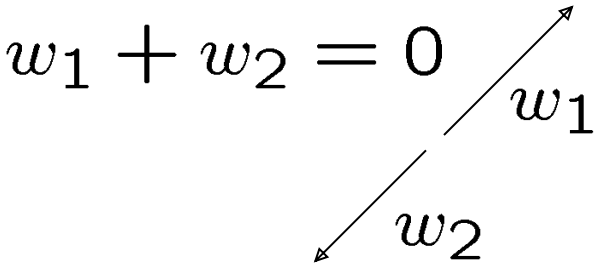
$$w_1, w_2, \ldots, w_n$$

When these n vectors are linearly independent

$$b_1 w_1 + b_2 w_2 + \cdots + b_n w_n = 0 \Leftrightarrow b_1 = b_2 = \cdots = b_n = 0$$



linearly independent



linearly dependent

Quantum Simon's Problem

Think about the bit strings s as vectors in $\mathbb{Z}_2^n$

Multiple runs of the quantum algorithm yield equations

$$\begin{aligned} y_1 \cdot s &= 0 \\ y_2 \cdot s &= 0 \\ &\vdots \\ y_k \cdot s &= 0 \end{aligned}$$

random uniform y_i

If we obtain $n-1$ linearly independent equations of this form,
we win (Gaussian elimination)

(Z2)n Vectors

$$y \cdot s = 0 \mod 2$$

Notice that if y is one of vectors with only one 1:

$$(0, 0, \ldots, 1, \ldots, 0) \cdot s = 0 \mod 2$$

i th bit

then this implies $s_i = 0$

Notice that if y is one of vectors with only one two 1's:

$$(0, 0, \ldots, 1, 0, \ldots, 0, 1) \cdot s = 0 \mod 2$$

then this implies $s_i + s_n = 0$ or $s_i = s_n$

(Z2)n Gaussian Eliminatio

$$y_1 \cdot s = 0$$
$$y_2 \cdot s = 0$$
$$\vdots$$
$$y_{n-1} \cdot s = 0$$

is equivalent to (remember, we know the y's)

$$(y_1)_1 s_1 + (y_1)_2 s_2 + \cdots + (y_1)_n s_n = 0 \mod 2$$
$$(y_2)_1 s_1 + (y_2)_2 s_2 + \cdots + (y_2)_n s_n = 0 \mod 2$$
$$\vdots$$
$$(y_{n-1})_1 s_1 + (y_{n-1})_2 s_2 + \cdots + (y_{n-1})_n s_n = 0 \mod 2$$

(Z2)n Gaussian Elimination

$$(y_1)_1 s_1 + (y_1)_2 s_2 + \cdots + (y_1)_n s_n = 0 \mod 2$$

$$+ \quad (y_2)_1 s_1 + (y_2)_2 s_2 + \cdots + (y_2)_n s_n = 0 \mod 2$$

$$((y_1)_1 + (y_2)_1) s_1 + ((y_1)_2 + (y_2)_2) s_2 + \cdots + ((y_1)_n + (y_2)_n) s_n = 0 \mod 2$$

We can add rows together to get new equations

We can always relabel the s_i and correspondingly $(y_j)_i$

(Z2)n Gaussian Elimination

Using these two techniques it is always possible to change the equations to the form:

$$(v_1)_1 s'_1 + (v_1)_n s'_n = 0 \mod 2$$

$$(v_2)_2 s'_2 + (v_2)_n s'_n = 0 \mod 2$$

$$(v_{n-1})_2 s'_{n-1} + (v_{n-1})_n s'_n = 0 \mod 2$$

Where the prime indicates that the s_i may have been permuted.

Depending on the v's this allows us to find the s_i

(Z2)n Gaussian Elimination

Example:

$$y_1 = (1, 0, 0, 1)$$
$$y_2 = (1, 1, 1, 1)$$
$$y_3 = (0, 0, 1, 1)$$

$$y_1 \cdot s = 0 \quad (1)$$
$$s_1 + s_4 = 0 \mod 2$$
$$y_2 \cdot s = 0 \quad (2)$$
$$s_1 + s_2 + s_3 + s_4 = 0 \mod 2$$
$$y_3 \cdot s = 0 \quad (3)$$
$$s_3 + s_4 = 0 \mod 2$$

$s_1 + s_4 = 0 \mod 2$ already in correct form

$s_2 + s_4 = 0 \mod 2$ add all three equations

$s_3 + s_4 = 0 \mod 2$ already in correct form

solutions:

$$s_4 = 0 \rightarrow s = (0, 0, 0, 0)$$
$$s_4 = 1 \rightarrow s = (1, 1, 1, 1)$$

Quantum Simon's Problem

Think about the bit strings s as vectors in $\mathbb{Z}_2^n$

Multiple runs of the quantum algorithm yield equations

$$\begin{aligned} y_0 \cdot s &= 0 \\ y_1 \cdot s &= 0 \\ &\vdots \\ y_k \cdot s &= 0 \end{aligned}$$

random uniform y_i

If we obtain n linearly independent equations of this form, we win (Gaussian elimination)

Suppose we have k linearly independent y_i 's. What is the probability that y_{k+1} is linearly independent of previous y_i 's?

$$Pr = \frac{2^n - 2^k}{2^n} = 1 - 2^{k-n}$$

Quantum Simon's Problem

$$Pr = \frac{2^n - 2^k}{2^n} = 1 - 2^{k-n}$$

What is the probability that our $n-1$ equations are linearly independent?

$$Pr(succ) = \left(1 - \frac{1}{2^n}\right)\left(1 - \frac{1}{2^{n-1}}\right)\cdots\left(1 - \frac{1}{4}\right)$$

$$= \prod_{k=1}^{n}\left(1 - \frac{1}{2^k}\right) > \prod_{k=2}^{\infty}\left(1 - \frac{1}{2^k}\right) \approx 0.28879$$

With constant probability we obtain linearly independence and hence solve Simon's problem.

Simon's Problem



Dan Simon

1994: Simon's Algorithm

Bounded error classical q. complexity: $\Omega(2^{n/2})$

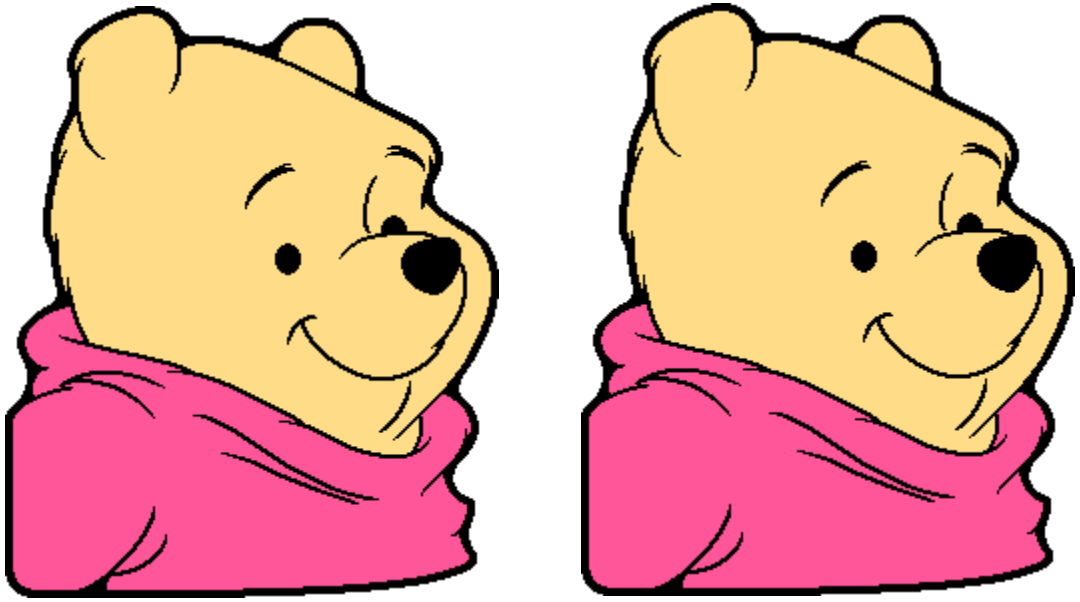
Bounded error quantum q. complexity: $O(n)$

(first exponential separation!)

Pooh-Pooh?

People like to pooh-pooh these early problems because they do not solve problems which are “natural”

This is silly. These results show that treating a device as classical or as quantum show amazing differences.



Killer Application?

In 1994, after Simon's algorithm, quantum computers were interesting, but were still a novelty.

Next Lecture we will see how this all changed, when Peter Shor discovered how to factor efficiently using a quantum algorithm.



Peter Shor